

Variables, Expressions, and Statements

CS 101 Algorithmic Problem Solving

Fall 2024

Before we begin with this reading, welcome everyone to CS 101. We hope you will have a great time with this course. Should you face any difficulty at any point, feel free to reach out to your instructors or TAs and they would be more than happy to help you out.

Where do we start?

If you've read the syllabus (which you should!), you'd know that later on in the course we will be introducing pseudocode and the Python programming language which you will be using to write your own algorithms to solve a slew of problems. While that might sound intimidating right now, an algorithm is no different than the instructions manual for building a LEGO set. It is just a chain of commands, telling a computer what, when, and how to perform certain actions. To write an algorithm, you need certain tools; some of which you will learn about in this reading.

Variables

A variable is something whose value can be changed, either by you or as a result of the instructions you give to your program. Assigning a value to a variable in Python is quite simple as shown in the code block below.

```
>>> n = 4
>>> pi = 3.142
```

In the above example, we've made two variables. The variable n has the value 4, and the variable π has the value 3.142. A variable name can be as long as you like and can contain numbers, and both upper case and lower case character. However, there are some rules when it comes to naming the variables in Python that you must remember.

- A variable name cannot start with a number. For example, *1piece* is an illegal variable name .
- A variable name cannot contain any special characters except underscore (`_`). For example, *dont@me* is an illegal variable name.
- A variable name cannot be the same as any of Python's keywords. For example, *class* is an illegal variable name.

While the first two rules are fairly straightforward to understand, the third rule might be a little confusing. Python has a list of keywords that the interpreter uses to recognize the structure of the program. These keywords cannot be used as variable names. On the next page is a list of keywords in the Python programming language.

<code>False</code>	<code>class</code>	<code>finally</code>	<code>is</code>	<code>return</code>
<code>None</code>	<code>continue</code>	<code>for</code>	<code>lambda</code>	<code>try</code>
<code>True</code>	<code>def</code>	<code>from</code>	<code>nonlocal</code>	<code>while</code>
<code>and</code>	<code>del</code>	<code>global</code>	<code>not</code>	<code>with</code>
<code>as</code>	<code>elif</code>	<code>if</code>	<code>or</code>	<code>yield</code>
<code>assert</code>	<code>else</code>	<code>import</code>	<code>pass</code>	
<code>break</code>	<code>except</code>	<code>in</code>	<code>raise</code>	

Generally, you should name your variables something meaningful, and not just random letters/words. You should also keep their names as brief as possible so that your code is easy to read. While variables on their own don't help us much, they are fundamental in providing instructions to the computer through expressions and statements.

Expressions

An expression is a combination of values, variables, and operators. A value or a variable by itself is also considered an expression.

```
>>> n
4
>>> 2 + pi
5.142
```

In the above example, we have two expressions. The first expression refers to the variable n we created in the previous section, and the second expression represents addition of 2 and the variable π we created in the previous section. Expressions can be used to represent all types of arithmetic functions, however their *syntax* (i.e. Python representation) might be different than what you are used to. Below is a non-exhaustive list of the Python syntax of some arithmetic functions that you will be regularly using. Note that any text written after the `#` symbol is a *comment*; comments are commonly used in programs to provide clarification to lines of code.

```
5 + 2 # addition
5 - 2 # subtraction
5 * 2 # multiplication
5 / 2 # division
5 ** 2 # exponentiation i.e. 5 raised to the power 2
5 // 2 # floor division i.e. rounding down the answer of 5 / 2 to the nearest
integer; so 5 // 2 equals 2
5 % 2 # modulo i.e. the remainder of 5 / 2; so 5 % 2 (read as 5 mod 2) equals 1
```

Expressions on their own are not very useful when it comes to writing algorithms, as nothing gets saved anywhere and they don't tell the program to output something; that is where statements come in.

Statements

Statements are units of code that have an effect like creating/updating a variable or outputting a value.

```
>>> n = 4
>>> pi = 3.142
>>> x = pi**2
>>> print(x)
9.872164
>>> pi = 3
>>> print(x)
9
```

In the above examples, we have six statements. The first three statements are assignment statements that assign a value to a variable. Note that the value of x is an expression which involves another variable, and hence, the value of x depends on the value of pi . The fourth and sixth statements are print statements. The first print statement displays the value of x , which at this point is 9.872164 (as $x = 3.142**2$). The fifth statement is an assignment statement that updates the value of pi which in turn updates the value of x as x depends on pi . The second print statement hence displays 9.

Order of operations

When it comes to evaluating an expression that contains more than one operator, Python follows the mathematical convention of **PEMDAS**.

- **P**arentheses have the highest precedence and can be used to force an expression to evaluate in the order you want. You can also use parentheses to make an expression easier to read.
- **E**xponentiation has the next highest precedence.
- **M**ultiplication and **D**ivision have the same and the next highest precedence.
- **A**ddition and **S**ubtraction have the same and the lowest precedence.
- In case of operators with same precedence, they are evaluated from left to right (except in the case of exponentiation, which is evaluated from right to left). For example, in the expression $3/4*2$, the division happens before the multiplication. You can use parentheses if you wish to perform multiplication first as follows $3/(4*2)$.